Case Study—From NAND to XOR to Half-Adder

This chapter is a worked case study in closure under composition: starting from a single primitive gate, NAND, we derive the familiar basis $\neg$, $\wedge$, and $\vee$, then build XOR and finally a half-adder. The point is not merely that these constructions are possible, but that the resulting implementations carry observable artifacts—depth, size, and timing behavior—that matter when the same Boolean specification is realized in hardware.

We will keep two views in play throughout. At the specification level, a circuit denotes a Boolean function and may exhibit invariances such as commutativity under swapping inputs. At the implementation level, the same function can be realized by many structurally different netlists, each with different depth, gate count, fanout structure, and sensitivity to timing-window violations. In this sense, the chapter illustrates how semantic equivalence can coexist with distinct physical behavior.

A useful error metric for this setting is the gate-level timing-window violation rate, denoted $\varepsilon(r)$. Informally, $\varepsilon(r)$ measures the fraction of runs in which a circuit exhibits a hazard or incorrect transient behavior under a given fault or timing regime r. We will refer to $\varepsilon(r)$ when discussing hazards and fault sensitivity, especially for derived implementations whose logical correctness does not by itself guarantee robust behavior.

Starting from NAND, one obtains negation by tying the inputs together:

$$\neg a \equiv a \uparrow a,$$

where $\uparrow$ denotes NAND. From this, conjunction and disjunction can be expressed in the usual way:

$$a \wedge b \equiv \neg(a \uparrow b), \qquad a \vee b \equiv (\neg a) \uparrow (\neg b).$$

These identities show that NAND is functionally complete. They also expose a basic depth/size trade-off: a direct specification may be compact as a formula, while a NAND-only implementation expands into multiple primitive gates. Different decompositions can reduce depth at the cost of additional gates, or reduce size at the cost of longer critical paths.

The XOR gate is a particularly instructive example because it admits several equivalent forms and is sensitive to implementation choice. One standard decomposition is

$$a \oplus b \equiv (a \wedge \neg b) \vee (\neg a \wedge b).$$

This expression makes the symmetry of XOR explicit: swapping a and b leaves the function unchanged. Other algebraically equivalent forms are often preferable in hardware synthesis, depending on the available primitives and optimization objective. For example, XOR can also be written using NAND-only subcircuits by substituting the derived forms of $\neg$, $\wedge$, and $\vee$, though the resulting netlist may differ substantially in depth and hazard profile. In practice, one may choose between a compact but deeper realization and a flatter realization with more shared structure, depending on the target timing constraints.

The half-adder combines XOR and AND to produce a sum bit and a carry bit. Its specification is

$$\mathrm{sum}(a, b) = a \oplus b, \qquad \mathrm{carry}(a, b) = a \wedge b.$$

As a Boolean relation, the half-adder is invariant under swapping the inputs a and b: both outputs are unchanged by the transposition $(a, b) \mapsto (b, a)$. This invariance is a semantic property of the specification, not of any particular gate-level realization. A NAND-based implementation may preserve the same input symmetry only up to circuit isomorphism; the physical netlist can still privilege one input path over another, affecting delay and transient behavior.

This distinction between specification and artifact is central to the case study. Two implementations can compute the same half-adder truth table while differing in gate count, logic depth, reconvergent fanout, and susceptibility to hazards. In particular, a design that is logically correct may still exhibit a larger $\varepsilon(r)$ under a fault model that perturbs arrival times or introduces brief glitches. Thus, correctness at the Boolean level is necessary but not sufficient for robust implementation.

A compact NAND-only realization of the half-adder can be obtained by first building $\neg$, $\wedge$, and $\vee$, then assembling XOR and carry from those derived blocks. This modular route is easy to reason about and aligns with the compositional perspective developed earlier in the book. However, synthesis tools may choose alternative factorizations that reduce depth or exploit shared subexpressions. The resulting artifact is therefore best understood as a point in a design space rather than as a unique canonical form.

For reproducibility, it is useful to pair the logical derivation with an executable artifact. A minimal Verilog description can encode the NAND-derived half-adder, and a testbench can enumerate all four input combinations to confirm the truth table. The same harness can be extended with fault injection or timing perturbations to estimate $\varepsilon(r)$ under specified conditions. In that setting, the testbench serves not only as a correctness check but also as a measurement instrument for implementation artifacts.

```
module half_adder_nand (
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);
    wire n1, n2, n3, n4, n5;

    nand (n1, a, b);
    nand (n2, a, n1);
    nand (n3, b, n1);
    nand (sum, n2, n3);
    nand (carry, n1, n1);
endmodule

module half_adder_nand_tb;
    reg a, b;
    wire sum, carry;
    integer i;

    half_adder_nand dut (.a(a), .b(b), .sum(sum), .carry(carry));

    initial begin
        for (i = 0; i < 4; i = i + 1) begin
            {a, b} = i[1:0];
            #1;
            $display("a=%b b=%b | sum=%b carry=%b", a, b, sum, carry);
        end
        $finish;
```

```
    end
endmodule
```

The chapter's working conclusion is that NAND closure gives a complete algebraic basis for Boolean construction, but the path from basis to system is not neutral. Each derivation choice affects depth, size, symmetry realization, and hazard behavior. The half-adder is therefore a small but representative example of the book's broader theme: compositional equivalence at the semantic level does not erase the structure of the implementation that realizes it.

In summary, the case study provides a concrete bridge from the abstract closure results of the earlier chapters to the implementation-sensitive concerns that recur later in the book. The same Boolean function can be presented as an equation, a netlist, or a testable artifact, and each presentation reveals different invariants and different failure modes. That multiplicity is not a defect; it is the central phenomenon the book aims to formalize.